

E& - Churn Model Use Case

This document provides a detailed, technical, and expansive explanation of the methodology employed in developing a customer churn prediction model, based on the analysis and work done within the E&_churn. The process is broken down into three critical phases: **Feature Engineering**, **Model Choice and Selection**, and the structured **Training Process**. The goal of this machine learning initiative is to accurately predict which customers are likely to churn, allowing the telecommunications company to deploy targeted retention strategies.

1. Feature Engineering: Transforming Raw Data into Predictive Signals

Feature engineering is arguably the most crucial step in any machine learning pipeline, as it involves transforming raw data into a format that a model can effectively learn from. Given that the underlying dataset is a typical Telco customer information sheet, it contains a mixture of numerical, binary, and multi-class categorical variables, necessitating a comprehensive preprocessing strategy to handle data types, missing values, and scale differences.

1.1. Data Cleaning and Imputation Strategy

The initial inspection of the dataset revealed 7,043 customer records and 21 distinct features. A key data quality issue was identified in the Total_Charges column.

Addressing Total_Charges

Initially, Total_Charges was loaded as an object (string) data type rather than a numerical type, suggesting the presence of non-numeric values. This is a common issue in real-world datasets where records for new customers (with tenure of 0 months) contain an empty string or a space instead of a zero or an imputed value for the cumulative charges.

1. **Type Conversion and Coercion:** The first step was to forcibly convert the Total_Charges column to a float type using `pd.to_numeric(..., errors='coerce')`. The `errors='coerce'` argument is vital here, as it automatically converts any non-numeric strings (the problematic missing values) into NaN (Not a Number), making them manageable for standard imputation techniques.
2. **Imputation Choice:** After coercion, a small number of missing values (11 instances) were confirmed in Total_Charges. Generally a small fraction of the total dataset (less than 0.2%), dropping the records is an option, but imputation is generally safer to retain maximum data utility, particularly if the missingness is non-random. The chosen imputation method was **median imputation**. The median is preferred over the mean for features like charges, as it is robust against potential outliers, ensuring that the few long-term, high-cost

customers do not unduly skew the imputed value. This method effectively handles the missingness while maintaining the central tendency and distribution characteristics of the feature.

1.2. Handling Class Imbalance

The churn prediction task is a binary classification problem, where the target variable, Churn, was successfully mapped to 1 (Yes) and 0 (No). A crucial finding in the data exploration phase was the significant class imbalance.

- **Non-Churn (Class 0):** Approximately 73.5% of the observations.
- **Churn (Class 1):** Approximately 26.5% of the observations.

This disparity is significant. A naïve model could achieve 73.5% accuracy simply by predicting 'No Churn' for every customer. This imbalance directly influenced the subsequent model choice and, most importantly, the optimization metric used during the training process, as detailed in Section 3. The awareness of this imbalance is a foundational element that dictates the focus on metrics beyond simple accuracy, such as Recall for the minority class (churners).

1.3. Categorical Feature Encoding

Machine learning models, particularly Support Vector Machines (SVMs), operate mathematically and require all input features to be numerical. The dataset contained a large number of object type columns representing categorical data, necessitating encoding.

Label Encoding for Binary Variables

For features with only two unique values (binary), **Label Encoding** was applied. This method converts the two categories directly into 0 and 1, which is computationally efficient and introduces no risk of inflating the dimensionality of the feature space. While the numeric assignment of 0 and 1 technically imposes an arbitrary order, for genuinely binary variables, this is typically accepted, especially when the magnitude does not heavily influence the model's interpretation of the variable (which is further mitigated by the kernel function in SVC). The gender column, for instance, is a perfect candidate for this simple transformation. Similarly, features like Senior_Citizen, Is_Married, Dependents, Phone_Service, and Paperless_Billing, which are binary, benefit from this method.

One-Hot Encoding (OHE) for Multi-Class Variables

For categorical features with three or more unique values, **One-Hot Encoding** was employed. This is crucial to prevent the model from misinterpreting arbitrary numerical relationships. If a feature like Contract (with 'Month-to-month', 'One year', 'Two year') were

simply encoded as 1, 2, and 3, the model would incorrectly infer that 'Two year' (3) is somehow numerically "three times better" than 'Month-to-month' (1).

OHE resolves this by creating a new binary column for *each* category. For example, the Internet_Service feature (with values like 'DSL', 'Fiber optic', 'No') would result in three new binary columns: Internet_Service_DSL, Internet_Service_Fiber optic, and Internet_Service_No.

- **Mitigation of the Dummy Variable Trap:** To avoid the **Dummy Variable Trap** (perfect multicollinearity), which can destabilize models, it is standard practice to drop one of the resultant binary columns from the OHE set. The information from the dropped category is implicitly captured when all other columns are 0.

Key features transformed using OHE included: Internet_Service, Contract, Payment_Method, and various service features such as Online_Security, Online_Backup, Device_Protection, Tech_Support, Streaming_TV, and Streaming_Movies.

1.4. Numerical Feature Scaling

The three core numerical features are tenure, Monthly_Charges, and Total_Charges. A review of the statistics shows these features operate on vastly different scales: tenure ranges from 0 to 72, Monthly_Charges up to 118.75, and Total_Charges up to 8684.80.

Standard Scaling Rationale

The model selected, a Support Vector Classifier (SVC), is a **distance-based algorithm**. Distance-based models are highly sensitive to the magnitude of the features. A feature with a large numerical range, such as Total_Charges, would numerically dominate the distance calculation (e.g., in the RBF kernel calculation), regardless of its actual predictive power, effectively overshadowing features with smaller ranges like tenure.

To ensure all features contribute equitably to the model's learning process, **Standard Scaling** (StandardScaler) was applied to the numerical columns. This technique transforms the data such that the resulting distribution has a mean (μ) of 0 and a standard deviation (σ) of 1. The transformation formula is:

$$z = \frac{x - \mu}{\sigma}$$

This standardization centers the data and scales the features to a comparable range, which is mandatory for the robust performance of the SVC algorithm.

1.5. Feature Exclusion

Finally, the customerID column was identified as a unique identifier for each observation. This column holds no predictive information related to the likelihood of churn, and its

inclusion would only add noise and dimensionality to the model. Therefore, it was properly dropped from the feature matrix before model training.

2. Model Choice: Selection and Justification of Support Vector Classification

The core task is customer churn prediction, a classic binary classification problem. The choice of model is pivotal, and the notebook clearly indicates the selection of a **Support Vector Classifier (SVC)**.

2.1. Why Support Vector Classification?

SVC is a highly effective and theoretically sound supervised learning model for classification tasks, particularly well-suited for high-dimensional data and problems where the decision boundary is non-linear.

- **Optimal Separating Hyperplane:** The primary objective of an SVC is to find the **optimal separating hyperplane** that maximizes the margin (the distance between the hyperplane and the nearest data points, known as the support vectors). Maximizing this margin leads to better generalization capability and a lower risk of overfitting to the training data.
- **Effective in High-Dimensional Spaces:** The comprehensive feature engineering process, especially One-Hot Encoding, results in a significantly high-dimensional feature space. SVC performs robustly even when the number of dimensions exceeds the number of samples, a scenario where other models might struggle.
- **Handling Non-Linearity via the Kernel Trick:** Real-world customer behavior data, like the Telco churn dataset, is rarely linearly separable. SVC excels here due to the **Kernel Trick**.

2.2. The Radial Basis Function (RBF) Kernel

The notebook specified the use of the `kernel='rbf'` during hyperparameter tuning. The RBF kernel, often referred to as the Gaussian kernel, is the most popular choice for non-linear SVC models.

The RBF kernel operates by implicitly mapping the original data into a much higher-dimensional feature space, where it is more likely that a linear decision boundary (a hyperplane) can effectively separate the classes. The RBF kernel's equation is:

$$K(x_i, x_j) = \exp(-\gamma \|x_i - x_j\|^2)$$

This function calculates the similarity (or distance) between two data points (x_i and x_j) in the high-dimensional space without ever having to explicitly calculate the coordinates of the data in that space. This computational shortcut is the "trick."

The selection of the RBF kernel over a linear kernel is a strong indication that the model developer anticipated or confirmed complex, non-linear relationships between the customer features (e.g., tenure, services, contract type) and the target churn outcome.

2.3. Hyperparameter Significance

The performance of an RBF-kernel SVC is critically dependent on two hyperparameters, which were targeted for optimization via GridSearchCV:

1. **C (Regularization Parameter):** This parameter controls the trade-off between achieving a large margin and minimizing the classification error on the training data.
 - **Small C:** Encourages a wider margin, accepting more misclassifications (higher bias, lower variance). This creates a simpler decision boundary.
 - **Large C:** Penalizes misclassification errors heavily, forcing the model to classify training points correctly, potentially leading to a narrower margin and a higher risk of overfitting (lower bias, higher variance).
2. **Gamma (γ):** This parameter defines the influence of a single training example on the decision boundary. It is an inverse measure of the radius of influence of the samples selected by the model as support vectors.
 - **Small γ :** Implies a large radius of influence, resulting in a very smooth, less complex decision boundary.
 - **Large γ :** Implies a small radius of influence, resulting in a highly complex, possibly contorted decision boundary that attempts to perfectly fit every training data point, often leading to overfitting.

The careful tuning of C and gamma is essential to balance the bias-variance trade-off and achieve the best generalization performance.

3. Training Process: Strategy, Optimization, and Evaluation

The training process was meticulously designed to handle the data characteristics, particularly the class imbalance, ensuring the final model is both accurate and fit for the business objective (churn retention).

3.1. Data Partitioning

Before any model training or hyperparameter search, the feature matrix (X) and the target vector (y) were separated and then partitioned into training and testing sets.

- **Train/Test Split:** A standard 70% to 30% split was used, dedicating the larger portion for training and the remaining 30% for independent, unbiased evaluation (the testing set).

- **Stratified Sampling:** Crucially, the split was performed using **stratified sampling** (e.g., by setting `stratify=y` in `train_test_split`). Given the significant class imbalance (73.5% 'No', 26.5% 'Yes'), standard random sampling could lead to test sets having a disproportionately high or low ratio of churners. Stratified sampling ensures that the class distribution of the target variable (`y`) is maintained proportionally in both the training and testing sets, thereby providing a more representative and stable test-set evaluation.

3.2. Hyperparameter Optimization with GridSearchCV

To systematically find the optimal combination of the `C` and `gamma` hyperparameters, the **Grid Search with Cross-Validation (GridSearchCV)** method was employed.

Grid Definition

The search space (the grid) was defined by exploring a logarithmic range of values for `C` and `gamma`:

- `C`: [0.1, 1, 10, 100].
- `gamma`: [1, 0.1, 0.01, 0.001].
- `kernel`: ['rbf'] (Held constant, as RBF was the kernel of choice).

This search explores $4 \times 4 = 16$ distinct parameter combinations, systematically testing the model's performance under varying degrees of regularization and feature influence.

5-Fold Cross-Validation (CV)

Within `GridSearchCV`, **5-fold cross-validation (`cv=5`)** was implemented. This technique divides the training set into five equal sub-samples (folds). The model is trained and validated five times: in each iteration, four folds are used for training, and the remaining single fold is used for validation. The results are then averaged. This process:

1. **Reduces Variance:** Provides a more robust estimate of the model's performance by testing it across different subsets of the data.
2. **Mitigates Overfitting:** Ensures the chosen hyperparameters are not merely optimized for one specific random split of the training data.

3.3. Selection Metric: Prioritizing Recall

The most crucial design choice in the training process, driven by the class imbalance, was the selection of the optimization metric: **`scoring='recall'`**.

Rationale for Recall over Accuracy

In a churn prediction context, a business has two types of classification errors:

1. **False Positive (FP):** Predicting a customer will churn when they actually won't. Cost: Unnecessary, high-cost retention effort (e.g., discount offer).
2. **False Negative (FN):** Predicting a customer will *not* churn when they actually *will*. Cost: The loss of an actual customer and their lifetime value (LTV), which is typically a much higher business cost.

Recall, also known as the True Positive Rate or Sensitivity, is the proportion of actual positive cases (actual churners) that are correctly identified. It is calculated as:

$$\text{Recall} = \frac{\text{True Positives}}{\text{True Positives} + \text{False Negatives}}$$

By setting `scoring='recall'`, `GridSearchCV` explicitly prioritizes models that minimize False Negatives, meaning the optimal model is the one best at *catching* the actual churners. This directly aligns with the business objective of a churn model: identifying customers at risk so that preventative action can be taken, making Recall the superior metric for optimization in this domain.

3.4. Final Model and Performance

The Grid Search identified the best-performing hyperparameters that maximized Recall on the training data via 5-fold CV.

- **Best Parameters:** {'C': 100, 'gamma': 0.001, 'kernel': 'rbf'}.
- **Interpretation of Optimal Parameters:**
 - The relatively **high value of C (100)** indicates the model is willing to accept a more complex, narrower margin to reduce classification errors, suggesting that the data points near the decision boundary are critical for separation.
 - The **low value of γ (0.001)** indicates a broad, smoother decision boundary, suggesting that the optimal separation does not require a highly intricate, high-variance boundary, which helps in generalizing well to unseen data.

The final `best_model` was trained on the entire training set using these optimal parameters.

Evaluation and Interpretation

The model was finally evaluated on the hold-out test set (the 30% unseen data). The complete evaluation included:

1. **Classification Report:** Provides a holistic view of the model's performance, including Precision, Recall, and F1-score for both the 'No Churn' (0) and 'Churn' (1) classes.

2. **Confusion Matrix:** A 2×2 matrix summarizing the counts of True Positives, True Negatives, False Positives, and False Negatives, which are the raw numbers needed for business analysis.
3. **Accuracy Score:** Provides a general measure of overall correctness, though less informative than Recall in this imbalanced context.

The **Learning Curve Plot** was generated to provide a final diagnostic check on the model's generalization capabilities. A well-behaved learning curve—where the training and cross-validation scores converge at a high score—confirms that the model (SVC with RBF kernel) has achieved an appropriate balance between bias (underfitting) and variance (overfitting).

The process concluded with the final, optimized model being persisted to disk as `svm_churn_model.pkl` using `joblib`. This saving process allows the model to be easily deployed for real-time scoring of new customer data, completing the entire machine learning lifecycle from data acquisition to deployable asset. The careful attention paid to feature engineering and the strategic prioritization of the Recall metric ensure that the resulting model is highly effective for its intended business purpose: reducing customer attrition by proactively identifying those most likely to leave.